

Лекція 3

**Потоки введення-виведення
Серіалізація/десеріалізація
XML та JSON. Jackson
Regular Expressions**

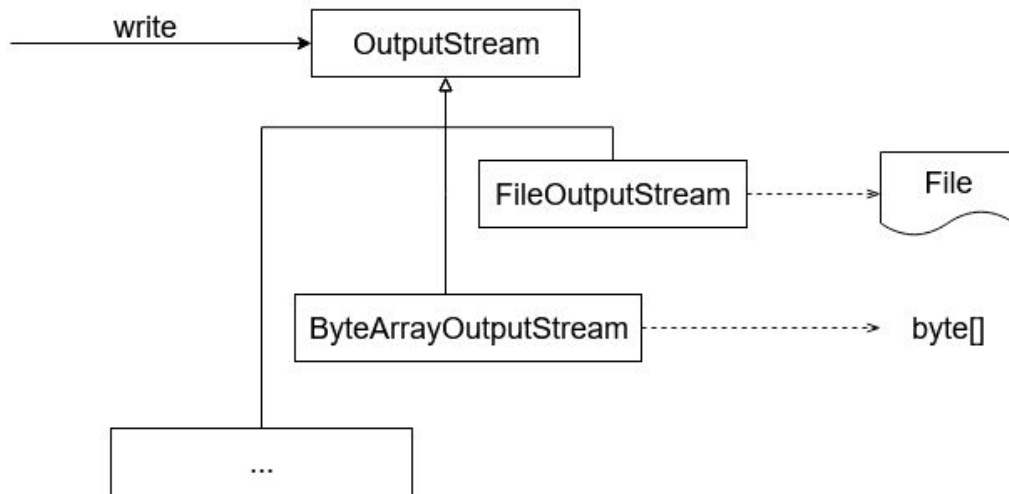
Поняття потоків введення/виведення

Потоком введення/виведення (I/O Stream) називають довільне джерело або приймач, що здатний генерувати або ж отримувати деякі дані.

Всі потоки поводять себе **однаковим чином**, хоча фізичні пристрої, з якими вони пов'язані, можуть сильно відрізнятися.

Реалізація конкретним потоком низькорівневого **способу приймання/передачі** інформації прихована від клієнтського коду.

OutputStream

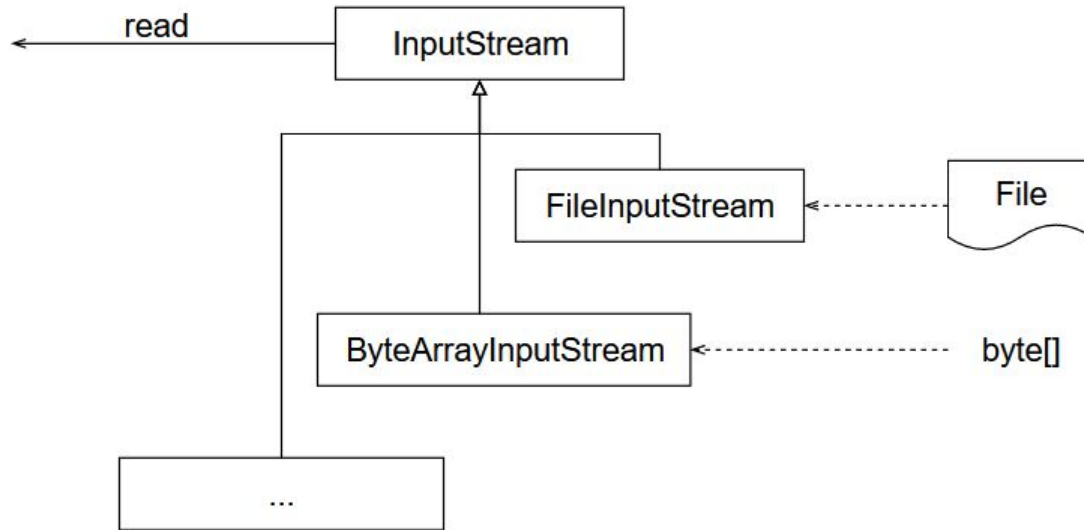


Пишемо у файл

```
byte[] testData = new byte[]{1, 2, 3, 4, 5};  
try (OutputStream out = new  
    FileOutputStream("test.txt")) {  
    out.write(testData);  
}
```

try-with-resources - автоматично закрий ресурс, тобто викличе метод `close()` у об'єкта `out`.

InputStream



Читаємо з (невеликого) файлу

```
private static byte[] readFile(String filename)
                                throws IOException {
    try (InputStream io = new FileInputStream(filename)) {
        int size = io.available();
        byte[] result = new byte[size];
        io.read(result, 0, size);
        return result;
    }
}
```

```
byte[] content = readFile("test.dat");
```

Копіюємо файли відповідально

```
String fromFileName = "from.jpg";
String toFileName = "to.jpg";

// використовуємо try-with-resources, щоб всі потоки точно
// були закриті
try (InputStream in = new FileInputStream(fromFileName);
    OutputStream out = new FileOutputStream(toFileName)) {
    // використовуємо буфер: це швидше, ніж писати/читати побайтно,
    // і не перенавантажує heap
    byte[] buffer = new byte[8 * 1024];
    int n; // в n буде записано, скільки ми реально прочитали
    while (-1 != (n = in.read(buffer))) {
        out.write(buffer, 0, n);
    }
}
```

Декорація (надбудова)

В Java для роботи з потоками існують класи, що реалізують **шаблон декорації**. Усі класи декоратори мають загальний базовий клас, для потоків введення - це **FilterInputStream**.

Класи декоратори мають один і той самий інтерфейс для роботи з потоками - **FilterInputStream**, що наслідує **InputStream**.

FilterInputStream агрегує в собі **InputStream** і всі свої методи делегує цьому внутрішньому об'єкту.

InputStream in =

```
new ZipInputStream(  
    new BufferedInputStream(  
        new FileInputStream("data.txt"))));
```


Декоратори для OutputStream

Декораторами для **OutputStream** є наслідники абстрактного класу **FilterOutputStream**.

PrintStream – додає можливість перетворення примітивних типів даних у послідовність байтів. Робить це за допомогою перевантаженого методу `print()`, який перетворює і поміщає дані у вихідний потік. Метод `print()` ніколи не генерує виключення `IOException`, а записує помилки у внутрішні змінні, які перевіряються методом `checkError()`.

BufferedOutputStream – буферизований вихідний потік. Прискорює виведення, якщо писати невеликими порціями.

DataOutputStream - потік для виведення значень примітивних типів. Має такі методи як `writeBoolean()`, `writeInt()`, `writeLong()`, `writeFloat()` тощо.

Архівування

Працює на базі механізму `Input/OutputStream` з використанням принципу вкладеності.

`ZipOutputStream`

`ZipInputStream`

`GZIPOutputStream`

`GZIPInputStream`

ZipOutputStream

```
try (ZipOutputStream out = new ZipOutputStream(  
    new FileOutputStream("archive.zip"))) {  
    pack(Paths.get("111.txt"), out);  
    pack(Paths.get("222.txt"), out);  
}
```

```
static void pack(Path source, ZipOutputStream out)  
    throws IOException {  
    // присвоїти ім'я файлу в архіві  
    out.putNextEntry(  
        new ZipEntry(source.getFileName().toString()));  
    // передати файл в потік архіву  
    Files.copy(source, out);  
}
```

ZipInputStream

```
try (ZipInputStream in = new ZipInputStream(  
    new FileInputStream("archive.zip"))){  
    ZipEntry entry;  
    while ((entry = in.getNextEntry()) != null) {  
        Path target = Paths.get(entry.getName());  
        Files.copy(in, target);  
    }  
}
```

Символьні потоки

Для роботи з символьними потоками в Java існують два базових класи – **Reader** та **Writer**

Reader містить абстрактні методи `read(...)` та `close()`. Додаткові методи оголошені в наслідниках цього класу.

Writer містить абстрактні методи `write(...)`, `flush()` та `close()`.

Деякі наслідники класу **Writer**

BufferedWriter - буферізований вивідний потік. Розмір буферу можна змінювати, хоча розмір за замовчуванням прийнятний для виконання більшості завдань.

CharArrayWriter - дозволяє виводити символи в масив як в потік.

PrintWriter - потік, забезпечений операторами `print()` та `println()`.

OutputStreamWriter – міст між класом **OutputStream** та класом **Writer**. Символи, що записані в цей потік, перетворюються на байти. При цьому можна обирати спосіб кодування символів.

FileWriter - потік для запису символів напямую у файл.

Наслідники класу Reader

BufferedReader - буферизований вхідний потік символів.

CharArrayReader - дозволяє читати символи з масиву як із потоку.

StringReader - те ж із рядку.

InputStreamReader – за допомогою методів класу Reader зчитує байти з потоку InputStream і перетворює їх на символи. В процесі перетворення використовує різні системи кодування.

FileReader - потік для читання символів із файлу.

Вводити рядки з клавіатури і писати їх у файл

// Створюємо буферизований символний вхідний потік

// Використовуємо клас PrintWriter для виведення

```
try (BufferedReader in = new BufferedReader(  
    new InputStreamReader(System.in));  
    PrintWriter out = new PrintWriter(  
        new FileWriter("out.txt"))) {
```

```
    while (true) {
```

```
        String s = in.readLine();
```

```
        if ("stop".equals(s)) // за словом stop зупинка
```

```
            break;
```

```
        out.println(s);
```

```
    }
```

```
}
```


Scanner

Дозволяє парсити вхідні потоки, виділяючи з них підрядки за рядком-розділювачем або regexp.

```
try (InputStream is = new FileInputStream("test.txt");
    Scanner scanner = new Scanner(is, "UTF-8")
        .useDelimiter(" ")) {

    while (scanner.hasNext()) {
        String row = scanner.next();
        System.out.println(row);
    }
}
```

RandomAccessFile

RandomAccessFile надає доступ до довільних місць файлів.

Для переміщення по файлу в **RandomAccessFile** застосовується метод **seek()**.

RandomAccessFile не приймає участь у розглянутій вище ієрархії, проте реалізує інтерфейси **DataInput** та **DataOutput** (ті ж, що реалізовані класами **DataInputStream** та **DataOutputStream**).

Приклад роботи з RandomAccessFile

```
RandomAccessFile rf = new RandomAccessFile("rtest.dat", "rw");  
// Записати до файлу 10 чисел та закрити файл  
for(int i = 0; i < 10; i++){  
    rf.writeDouble(i * 1.1);  
    rf.close();  
// Відкрити файл, записати до нього число зверху 5-го  
    rf = new RandomAccessFile("rtest.dat", "rw");  
    rf.seek(5 * 8);  
    rf.writeDouble(47.0001);  
    rf.close();  
// Відкрити файл з можливістю читання тільки "r"  
    rf = new RandomAccessFile("rtest.dat", "r");  
// Прочитати 10 чисел та вивести їх на екран  
    for(int i = 0; i < 10; i++){  
        System.out.println("Value " + i + ": " + rf.readDouble());  
        rf.close();  
    }  
}
```

Клас File

Клас **File** призначений для роботи з елементами файлової системи – каталогами та файлами.

Кожний об'єкт **File** являє собою абстрактний файл або каталог, можливо і не існуючий.

Абстрактний шлях, який включає в себе об'єкт **File**, складається з необов'язкового системно залежного префіксу і послідовності імен.

- префікс в різних операційних системах виглядає по-різному: "C:", "D:" в системі Windows, символ кореневого каталогу в системі Linux тощо.
- кожне ім'я послідовності є **ім'ям каталогу**, а остання ім'я може бути іменем каталогу або файлу.

Шлях може бути **абсолютним** або **відносним**.

Тека (папка) – це особливий `File`, який містить в собі список інших файлів та каталогів.

Для каталогу метод `isDirectory()` повертає **true**.

Метод `File[] listFiles()` повертає список підкаталогів і файлів даного каталогу.

Приклад: отримати масив файлів і каталогів, які знаходяться в робочому (або поточному) каталозі.

```
File path = new File(".");
File[] list = path.listFiles();
for(int i = 0; i < list.length; i++) {
    System.out.println(list[i].getName());
}
```

Класи Path та Files

Знаходяться в пакеті `java.nio.file`. Замінюють та доповнюють `java.io.File`.

Path - представлення шляху до файлу або теки.

Files - містить низку операцій для роботи з файлами:

- `copy(..)`
- `move(..)`
- `createDirectories(..)`
- `getLastModifiedTime(..)`
- `delete(..)`
- ...



Серіалізація

Серіалізація дозволяє перетворити об'єкт в потік байтів, щоб за необхідності повністю відновити об'єкт з такого потоку.

Серіалізація може використовуватися для:

- зберігання об'єктів в постійній пам'яті;
- передачі даних мережею.

Інтерфейс Serializable

Щоб володіти здатністю до серіалізації, клас повинен:

Реалізовувати інтерфейс **Serializable**

- інтерфейс Serializable не містить ніяких методів. Він просто слугує індикатором того, що клас може бути серіалізований.

```
public class MyClass implements Serializable{  
    ...  
}
```

Всі атрибути класу повинні серіалізуватися:

- атрибути **примітивних типів** серіалізуються за замовчуванням;
- якщо атрибут не повинен бути збережений при процесі серіалізації, для нього потрібно задати модифікатор **transient**
 - при серіалізації він буде проігнорований;
 - при десеріалізації значення цього атрибуту буде пустим.

Всі **підтипи** класу, що серіалізується, є такими, що серіалізуються.

Запис-читання об'єктів

Серіалізовані об'єкти можна записувати та зчитувати за допомогою класів `ObjectOutputStream` та `ObjectInputStream`.

void `writeObject(Object obj)` – записування об'єкту.

`Object readObject()` – читання об'єкту.

- метод може також генерувати `ClassNotFoundException`.

При **десеріалізації** об'єкту, він повертається у вигляді об'єкту класу `Object`. Для того щоб використовувати десеріалізований клас, необхідно здійснити явне його перетворення (приведення) до об'єкту потрібного типу.

Приклад серіалізації об'єктів

// Серіалізація

```
try (ObjectOutputStream oos = new ObjectOutputStream(  
    new FileOutputStream("state.bin"))) {  
  
    oos.writeDouble(3.14159265D);  
    oos.writeObject("The value of PI");  
    oos.writeObject(new Point(10, 253)); //запис об'єкту класу Point  
    oos.flush();  
}
```

// Десеріалізація

```
try (ObjectInputStream ois = new ObjectInputStream(  
    new FileInputStream("state.bin"))) {  
  
    System.out.println("Double: " + ois.readDouble());  
    System.out.println("String: " + ois.readObject().toString());  
    System.out.println("Point: " + (Point) ois.readObject());  
}
```

Недоліки класичної серіалізації в Java

Платформозалежна

- тільки Java визначеної версії.

При зміні класів (встановлена нова версія програми), можуть бути проблеми з десеріалізацією.

Все повинно бути Serializable.

Може створювати проблеми в безпеці програми

- <https://www.baeldung.com/java-deserialization-vulnerabilities>

Альтернатива: **JSON**, **XML**, CSV, etc.

XML (eXtensible Markup Language)

XML - розширювана мова розмітки

XML розроблений для зберігання та транспортування даних

XML розроблено таким чином, щоб його можна було читати як людиною, так і машиною

XML був розроблений, щоб бути самоописовим

XML є рекомендацією W3C

Приклад XML файла

```
<?xml version="1.0" encoding="UTF-8"?>
<bookstore>
  <book id="1" category="java">
    <title>Thinking in Java, 2nd Edition</title>
    <author>Bruce Ekkel</author>
    <publisher>O'Reilly</publisher>
    <year>2000</year>
    <price>48.50</price>
  </book>
  <book id="2">
    <title lang="en">Delphi 5</title>
    <author>S. Bobrovskij</author>
    <publisher>Piter</publisher>
    <year>2000</year>
    <price>35.00</price>
  </book>
</bookstore>
```

Правила синтаксису XML

Документи XML повинні мати кореневий елемент

Пролог XML

```
<?xml version="1.0" encoding="UTF-8"?>
```

Пролог XML необов'язковий. Якщо він існує, він має бути першим у документі.

Коментарі в XML

вірно

```
<!-- This is a comment -->
```

невірно

```
<!-- This is an invalid -- comment -->
```

Правила синтаксису XML

Значення атрибутів XML завжди повинні бути взяті в лапки

```
<title lang="en">Delphi 5</title>
```

Елементи XML мають бути належним чином вкладені

невірно

```
<b><i>This text is bold and italic</b></i>
```

вірно

```
<b><i>This text is bold and italic</i></b>
```

Правила синтаксису XML

Посилання на сутності (entity reference)

<	<	less than
>	>	greater than
&	&	ampersand
'	'	apostrophe
"	"	quotation mark

Приклад:

```
<message>salary &lt; 1000</message>
```




Правила іменування елементів XML

Імена елементів чутливі до регістру

Назви елементів **повинні** починатися з літери або підкреслення

Імена елементів **не можуть** починатися з літер xml (або XML, або Xml тощо)

Імена елементів **не можуть** містити пробіли

Простори імен XML (XML Namespaces)

Конфлікти імен

Постачальник 1:

```
<message>  
  <header>Main topic</header>  
  <body>loooooong text</body>  
</message>
```

Постачальник 2:

```
<message>  
  <thema>Main topic</thema>  
  <txt>loooooong text</txt>  
  <body>0111010110101100110</body>  
</message>
```

Простори імен XML (XML Namespaces)

Простір імен XML – атрибут **xmlns**

Простір імен може бути визначений атрибутом `xmlns` у початковому тегу елемента або в кореневому елементі XML.

Оголошення простору імен має такий синтаксис.

`xmlns:префікс = "URI"`

Простори імен XML (XML Namespaces)

Приклади:

```
<f:message xmlns:f="http://www.first.com/mess/">
  <f:header>Main topic</f:header>
  <f:body>loooooong text</f:body>
</f:message>
<s:message xmlns:s="http://www.second.com/message/">
  <s:thema>Main topic</s:thema>
  <s:txt>loooooong text</s:txt>
  <s:body>0111010110101100110</s:body>
</s:message>
```

або

```
<root xmlns:f="http://www.first.com/mess/"
      xmlns:s="http://www.second.com/message/">
  <f:message>
    <f:header>Main topic</f:header>
    <f:body>loooooong text</f:body>
  </f:message>
  <s:message>
    <s:thema>Main topic</s:thema>
    <s:txt>loooooong text</s:txt>
    <s:body>0111010110101100110</s:body>
  </s:message>
</root>
```

XML DTD (Document Type Definition)

DTD визначає структуру, доступні елементи та атрибути документа XML

DTD задається перед кореневим елементом. Може задаватися безпосередньо в документі xml, або як посилання на зовнішній файл (зазвичай з розширенням .dtd)

Приклад:

```
<?xml version="1.0" encoding="UTF-8"?>  
<!DOCTYPE root SYSTEM "Note.dtd">
```

або

```
<?xml version="1.0" encoding="UTF-8"?>  
<!DOCTYPE root  
[  
  <!ELEMENT root (message)>  
  <!ELEMENT message (header, body)>  
  <!ELEMENT header (#PCDATA)>  
  <!ELEMENT body (#PCDATA)>  

```

DTD для оголошення сутності

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE note [
    <!ENTITY nbsp "&#xA0;">
    <!ENTITY writer "Writer: Donald Duck.">
    <!ENTITY copyright "Copyright:W3Schools.">
]>
<note>
    <to>Tove</to>
    <from>Jani</from>
    <heading>Reminder</heading>
    <body>Don't forget me this weekend!</body>
    <footer>&writer;&nbsp;&copyright;</footer>
</note>
```

XML Schema (XSD)

XML Schema описує структуру документа XML, як і DTD і є альтернативою на основі XML.

```
<?xml version="1.0"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="root">
    <xs:complexType>
      <xs:sequence>
        <xs:element name="message">
          <xs:complexType>
            <xs:all>
              <xs:element name="header" type="xs:string"/>
              <xs:element name="body" type="xs:string"/>
            </xs:all>
          </xs:complexType>
        </xs:element>
      </xs:sequence>
    </xs:complexType>
  </xs:element>
</xs:schema>
```

Аналізатори SAX і StAX

SAX і StAX - потокові аналізатори на основі подій

SAX – переглядає XML та викликає методи наданого вами об'єкта обробника.

StAX – потрібно самим переміщати аналізатор StAX від елемента до елемента.

Переваги:

Мале споживання пам'яті: файл читається послідовно, не потрібно зберігати весь документ.

Швидка обробка великих файлів.

Недоліки:

Однонаправлений доступ: не можна переміщатися документом назад.

Складність використання: необхідно керувати станом програми вручну.

Аналізатор DOM

Аналізатор **DOM** створює об'єкти у пам'яті, які представляють усе дерево документа XML та повний стан набору інформації для документа XML.

Потрапивши в пам'ять, деревами **DOM** можна вільно переміщатися та довільно аналізувати їх, що забезпечує максимальну гнучкість для розробників.

DOM дає змогу створювати вузли, видаляти вузли, змінювати їхній вміст і проходити через ієрархію вузлів.

Ціна гнучкості є потенційно великий обсяг пам'яті та значні вимоги до процесора



JAXB (Java Architecture for XML Binding)

JAXB забезпечує швидкий і зручний спосіб маршалінга (запису) об'єктів Java в XML і демаршалінга (читання) XML в об'єкти.

JAXB підтримує framework, який відображає елементи та атрибути XML у поля та властивості Java за допомогою **анотацій** Java.

Анотації JAXB

```
@XmlElement(name = "messagestore")
public class MessageStore {
    @XmlElement(required = true)
    protected String category;
    @XmlElementWrapper(name = "messages")
    @XmlElement(name = "message")
    protected List<Message> messages;
    //getters and setters
}
```

@XmlElement : ім'я кореневого елемента XML

@XmlType: визначає порядок запису полів у файлі XML

@XmlElement: позначає елемента XML

@XmlAttribute: визначає, що поле id відображається як атрибут

@XmlTransient: анотуються поля, які ми не хочемо включати в XML

Маршалінг/Демаршалінг JAXB

// Маршалінг

```
MessageStore store = getTestMessageStore();  
JAXBContext context = JAXBContext.newInstance(MessageStore.class);  
Marshaller mar = context.createMarshaller();  
mar.setProperty(Marshaller.JAXB_FORMATTED_OUTPUT, Boolean.TRUE);  
mar.marshal(store, new File("./out/public_message.xml"));
```

// Демаршалінг

```
JAXBContext context = JAXBContext.newInstance(MessageStore.class);  
MessageStore store = (MessageStore) context.createUnmarshaller()  
    .unmarshal(new FileReader(source));
```

Складні типи даних JAXB

```
public class DateAdapter extends XmlAdapter<String, Date> {  
    private static final DateFormat dateFormat  
        = new SimpleDateFormat("yyyy-MM-dd HH:mm:ss");  
  
    @Override  
    public Date unmarshal(String v) throws Exception {  
        return dateFormat.parse(v);  
    }  
  
    @Override  
    public String marshal(Date v) throws Exception {  
        return dateFormat.format(v);  
    }  
}
```

Приклад використання:

```
@XmlJavaTypeAdapter(value = DateAdapter.class)  
protected Date createDate;
```

Результат:

```
<createDate>2016-11-10 23:44:18</createDate>
```

JSON (JavaScript Object Notation)

JSON - це текстовий формат, повністю незалежний від мови реалізації

- простий формат обміну даними
- зручним як для читання та написання людиною, так і для парсингу та генерації комп'ютером
- базується на підмножині мови програмування JavaScript (формат оголошення об'єкту в JavaScript)

JSON базується на двох структурах даних:

- Колекція пар ключ/значення.
- Упорядкований список значень.

Як значення в JSON можуть бути використані:

Об'єкт – `"object" : {"name": "Marley", "firstName": "Bob"}`

Масив – `"array": [1, "2", "3", {"some": "Object"}, 5]`

Число.

Літерали – `true`, `false` та `null`.

Рядок - `"Hello world!"`

Приклад файла JSON

```
{  
  "category" : "all",  
  "messages" : [ {  
    "header" : "All Header 1",  
    "body" : "All Body1",  
    "size" : 6,  
    "messid" : "0001"  
  }, {  
    "header" : "All Header 2",  
    "body" : "All Body2",  
    "size" : 6,  
    "messid" : "0002"  
  }, {  
    "header" : "All Header 3",  
    "body" : "All Body 3",  
    "size" : 7,  
    "messid" : "0003"  
  } ]  
}
```

Бібліотека Jackson

В pom.xml

```
<properties>
  <jackson.version>2.20.1</jackson.version>
</properties>
<dependency>
  <groupId>com.fasterxml.jackson.core</groupId>
  <artifactId>jackson-annotations</artifactId>
  <version>${jackson.version}</version>
</dependency>
<dependency>
  <groupId>com.fasterxml.jackson.core</groupId>
  <artifactId>jackson-core</artifactId>
  <version>${jackson.version}</version>
</dependency>
<dependency>
  <groupId>com.fasterxml.jackson.dataformat</groupId>
  <artifactId>jackson-dataformat-xml</artifactId>
  <version>${jackson.version}</version>
</dependency>
```


Аннотації Jackson

@JsonAutoDetect - Позначає клас як готовий до серіалізації в JSON.

@JsonIgnore - Властивість ігнорується під час серіалізації.

@JsonProperty - Дозволяє встановити інше ім'я поля під час серіалізації.

@JsonPropertyOrder - Ставиться перед класом. Дозволяє встановити порядок полів для серіалізації.

@JacksonXmlProperty - дозволяє контролювати деталі елемента, який відображається (визначити простір імен, локальну назву та те, чи поле серіалізовано як Element чи Attribute).

@JacksonXmlRootElement - має подібну роль до @JacksonXmlProperty, але для кореневого елемента всього документа.

@JacksonXmlText - вказує на те, що елемент має відтворюватися як простий текст без іншого елемента, який його містить.

@JacksonXmlCDATA - вказує на те, що навколо вмісту елемента слід розмістити оболонку CDATA.

@JacksonXmlElementWrapper - аналог @XmlElementWrapper з JAXB.

Анотації Jackson

```
@JsonAutoDetect

@JsonPropertyOrder({"bookname", "year", "annotation"})

public class Book {

    @JsonProperty("bookname")
    private String bookName;

    @JacksonXmlText
    @JacksonXmlCDATA
    private String annotation;
    private int year;
}

@JsonAutoDetect

@JacksonXmlRootElement(localName = "library")

public class Library {

    @JacksonXmlProperty(isAttribute = true, localName = "id")
    private String ident;
    private String name;

    @JacksonXmlElementWrapper(localName = "books")
    @JacksonXmlProperty(localName = "book")
    private List<Book> books = new ArrayList<>();

    @JsonIgnore
    private String location;
}
```

Анотації @JsonAnySetter та @JsonAnyGetter

```
private static class ExtendableBean {  
    public String name;  
  
    private Map<String, String> properties = new HashMap<>();  
  
    @JsonAnyGetter  
    public Map<String, String> getProperties() {  
        return properties;  
    }  
  
    @JsonAnySetter  
    public void add(String key, String value) {  
        properties.put(key, value);  
    }  
}  
  
public static void main(String[] args) throws Exception{  
    ExtendableBean bean = new ExtendableBean();  
  
    bean.name = "NAME Bean";  
  
    bean.add("attr1", "val1");  
  
    bean.add("attr2", "val2");  
  
    System.out.println(DEFAULT_MAPPER.writeValueAsString(bean));  
}
```

Результат

```
{ "name": "NAME Bean", "attr2": "val2", "attr1": "val1" }
```

Читання/запис за допомогою Jackson в JSON

//читання

```
ObjectMapper jsonMapper = new ObjectMapper();  
jsonMapper.enable(SerializationFeature.INDENT_OUTPUT);  
try {  
    Library l = jsonMapper.readValue(new File("./in/library_in.json"), Library.class);  
    System.out.println(l.toString());  
} catch (Exception e) {  
    throw new RuntimeException(e);  
}
```

//запис

```
ObjectMapper jsonMapper = new ObjectMapper();  
jsonMapper.enable(SerializationFeature.INDENT_OUTPUT);  
try {  
    jsonMapper.writeValue(new File("./out/library.json"), getLibrary());  
} catch (Exception e) {  
    throw new RuntimeException(e);  
}
```

Налаштування Jackson

```
private static final ObjectMapper DEFAULT_MAPPER;  
  
static {  
    ObjectMapper mapper = new ObjectMapper();  
    // Не створювати exception, якщо json має додаткові поля  
    // без серіалізації. Це корисно, коли ви хочете використовувати роjo  
    // для десеріалізації та дбає лише про частину json  
    mapper.configure(DeserializationFeature.FAIL_ON_UNKNOWN_PROPERTIES, false);  
    // Ігноруйте нульові значення під час запису json.  
    mapper.setSerializationInclusion(JsonInclude.Include.NON_NULL);  
    // Записувати час як рядок замість long, щоб його було зрозуміло людині.  
    mapper.configure(SerializationFeature.WRITE_DATES_AS_TIMESTAMPS, false);  
  
    mapper.registerModule(new JavaTimeModule());  
    DEFAULT_MAPPER = mapper;  
}
```

Jackson Streaming API

Streaming API читає та записує вміст JSON як окремі події. JsonParser читає дані, тоді як JsonGenerator записує дані. Це найпотужніший підхід з мінімальними накладними витратами та найшвидший в операціях читання/запису. Це аналог синтаксичного аналізатора Stax для XML.

JsonGenerator – записує JSON.

JsonParser – аналізує JSON.

Regular Expressions

Дозволяють задавати гнучкі правила порівняння строк

// ідентифікує мобільний номер

```
Pattern digitsPattern = Pattern.compile("\\+380\\d{9}");  
Matcher matcher = digitsPattern.matcher("+380685552244");  
System.out.println("Matches:" + matcher.matches());
```

Predefined character classes

- . Any character (may or may not match line terminators)
- \d** A digit: [0-9]
- \D** A non-digit: [^0-9]
- \s** A whitespace character: [\t\n\x0B\f\r]
- \S** A non-whitespace character: [^\s]
- \w** A word character: [a-zA-Z_0-9]
- \W** A non-word character: [^\w]

<https://regex101.com/>

<https://docs.oracle.com/en/java/javase/18/docs/api/java.base/java/util/regex/Pattern.html>

Greedy quantifiers

X?	X, once or not at all
X*	X, zero or more times
X+	X, one or more times
X{n}	X, exactly n times
X{n,}	X, at least n times
X{n,m}	X, at least n but not more than m times

<https://regex101.com/>

<https://docs.oracle.com/en/java/javase/18/docs/api/java.base/java/util/regex/Pattern.html>

Пошук значень в строках

```
List<String> log = List.of(
    "В групі A-1 знайдено 20 студентів",
    "В групі A-2 знайдено 22 студентів",
    "В групі B-1 знайдено 30 студентів");

Pattern logPattern = Pattern.compile(
    "В групі (\\S+) знайдено (\\d+) студентів");

for (String logItem : log) {
    Matcher matcher = logPattern.matcher(logItem);
    if (matcher.matches()) {
        String group = matcher.group(1);
        String count = matcher.group(2);
        System.out.println(group + ": " + count);
    }
}
```

Парсинг атрибутів в одинарних кавичках

```
List<String> transactionAttributes = List.of(
    "from='John Smith'",
    " to = 'Jane Smith'",
    "sum = '1000$' ",
    " comment = ''");

Pattern attributePattern =
    Pattern.compile("\\s*(\\S+)\\s*=\\s*'([^\']*)*'\\s*");

for (String logItem : transactionAttributes) {
    Matcher matcher = attributePattern.matcher(logItem);
    if (matcher.matches()) {
        String group = matcher.group(1);
        String count = matcher.group(2);
        System.out.println(group + ": \"" + count + "\"");
    }
}
```

Висновки

1. Завжди використовуйте **try-with-resources** коли працюєте з Closable-об'єктами (InputStream, OutputStream, etc.)
2. Уникайте завантаження цілих файлів в пам'ять будь-де, якщо це можливо (**використовуйте буферизацію**).
3. Не використовуйте Java-серіалізацію за можливості
4. Парсіль/генеріль json/xml/csv файли стандартними засобами, якщо це можливо, але не бійтеся парсити вручну, якщо підходить для задачі краще